

12/3/24

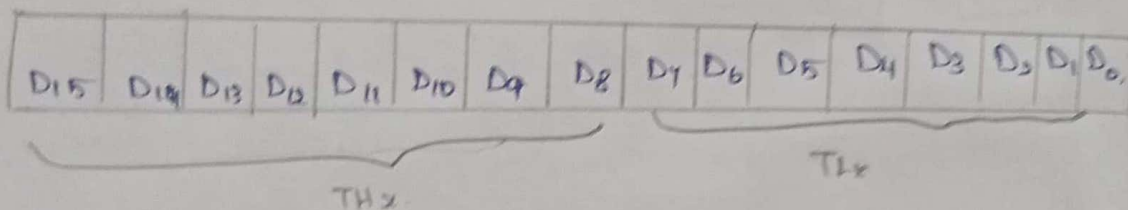
UNIT-5.

MICROCONTROLLER INTERFACING.

TIMERS IN 8051:

→ Both timers are 16 bit that can be accessed by 2 8 bit registers. (TLx, THx)

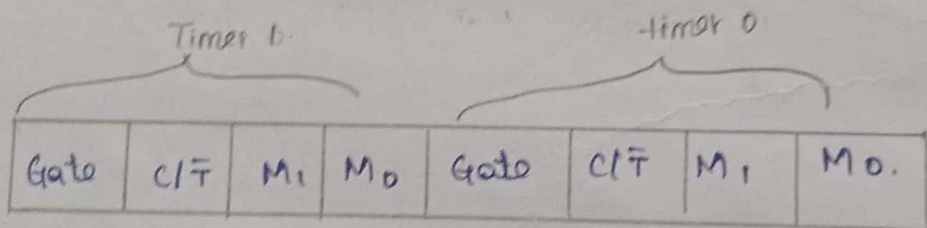
→ 2 timers — Timer 0
— Timer 1.



→ TMOD is 8 bit register

Most significant 4 bits for timer 1.

Least significant 4 bits for timer 0.



* Gate = if i use hardware to enable & disable the timer → 1.

if i use software instruction to enable & disable the timer → 0.

1 → counter.

0 → timer.

* M₁, M₀ - Mode bits.

range of values of timer

0 0 - mode 0 (13 bit timer mode) 0000-1FFF

0 1 - mode 1 (16 bit timer mode) 0000-FFFF

1 0 - mode 2 (8 bit timer with auto reload)

1 1 - mode 3 (split timer mode) { 00-FF (in T₁)
after overflow,
it starts from
[T_H]

timer 1 is used to stop

the activity, when timer 0 is used.

In mode 0, 1, 2 the timers independent,
whatever we can use.

In 8051, we have crystal oscillator it's
frequency is 11.0592 MHz.

* crystal oscillator freq = 11.0592 MHz

* each machine cycle contains 12 clk
cycles.

How much time taken
to complete full instruction
is 1 machine cycle.

* 1 machine cycle freq = $\frac{1}{12} \times 11.0592 \text{ MHz}$

$$f = 921.6 \text{ kHz.}$$

* 1 machine cycle time = $\frac{1}{921.6 \text{ K}}$

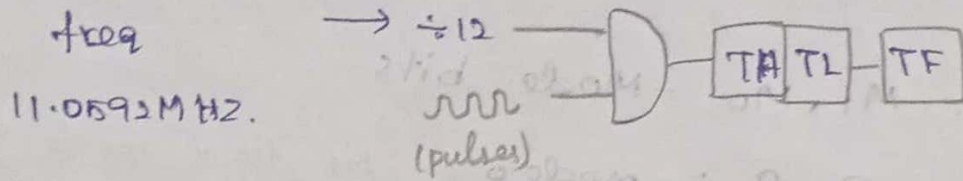
$$T = 1.085 \text{ } \mu\text{sec.}$$

(mode 0 & mode 1 is similar)

TF - timer flag. [it is set - when value changes
from FF to 00]

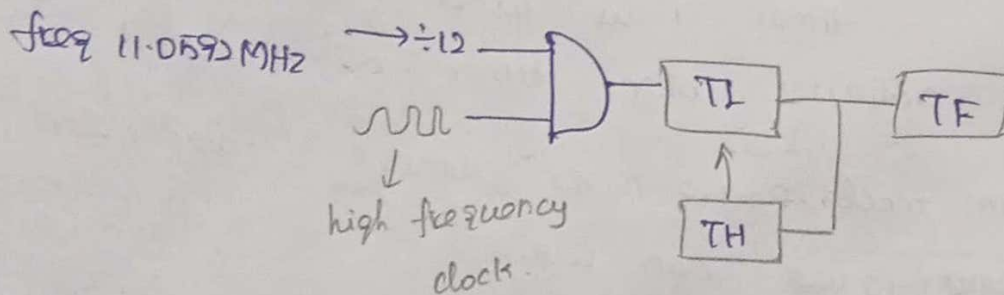
mode 1:

Crystal oscillator



mode 2:

Crystal oscillator



when i/p comes TL get increment after roll over it comes from TH, TF monitor this.

Steps for programming timer in Mode 1:

1. Load TMOD register with mode and timer 0 or 1.
2. Load TLx and THx with initial count corresponding to delay.
3. Start timer SET TR=0, SET TR=0.
4. Continuously monitor the time flag bit using instruction JNB TFx, target.

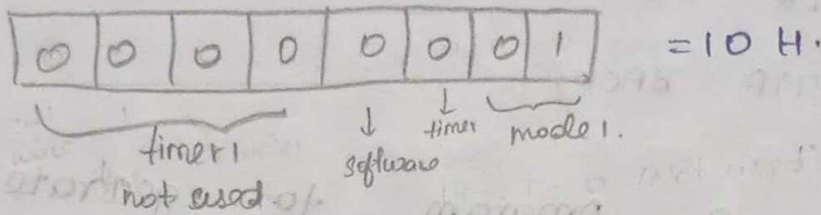
JNB $\rightarrow$ jump if bit not set.

5. When timer flag bit set stop the timer and clear the timer flag bit.

6. Go to step 2.

program to delay 10 msec:

Generate a delay of 10 ms using timer 0 in mode 1?



```
MOV TMOD, #01H
MOV TLO, #00H
MOV TH0, #DCH
SETB TR0
AGAIN: JNB TFO, AGAIN
CLR TFO
CLR TR0
```

$$\text{count} = \frac{10 \text{ ms}}{1.085 \mu\text{s}} = 9.216 \times 10^3 = 9216$$

FFFF → decimal equivalent.

$$65536 - \text{count} = 56320$$

↓
DCH
stored in TH
↓
TLO

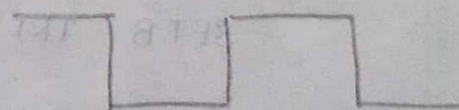
write a program to generate a square wave of 5 kHz on pin P.0

$$\text{delay} = 0.1 \text{ ms}$$

$$\text{count} = \frac{0.1 \text{ ms}}{1.085 \mu\text{s}} = 92.165$$

$$65536 - 92.16 (92) = 65444$$

↓
FFDCH
↓
TH TLO



$$T = \frac{1}{f} = \frac{1}{5 \times 10^3} = 0.2 \text{ ms}$$

$$\text{half time period} = \frac{0.2 \times 10^{-3}}{2}$$

$$= 0.1 \text{ ms}$$

```

MOV TMOD, #01H
BACK: MOV TLO, #AH
MOV TH0, #FF
SETB TRO
AGAIN: JNB TFO, AGAIN
CPL PI.0
CLR TFO
CLR TRO
SJMP BACK.

```

CPL → complement
if already 1, then it becomes 0

write a program to generate square wave delay of 5 μ sec using timer 1 & mode 2.

delay : 5 μ sec.

$$\text{count} = \frac{5 \times 10^{-6}}{1.085 \times 10^{-6}} = 4.608$$

$$256 - 4.6(5) = 251 \Rightarrow \text{FBH}$$

$\underbrace{\hspace{1cm}}_{\text{TH}}$

0	0	1	0	0	0	0	0
---	---	---	---	---	---	---	---

= 20H.

MOV TMOD, #20H. not used.

MOV TH1, #FBH

SETB TRI

HERE: JNB TFI, HERE

CLR TFI

CLR TRI

$$\begin{array}{r}
 2 \overline{) 251} \\
 \underline{125 -} \\
 62 - 1 \\
 \underline{31 -} 0 \\
 \underline{15 -} 1 \\
 \underline{7 -} 1 \\
 \underline{3 -} 1 \\
 \hline
 1 - 1
 \end{array}$$

11111011
F B

Serial Port programming in 8051.

Baud rate - no. of symbols transmitted per second.

Mc → Pc

crystal oscillator freq = 11.0593 MHz.

8051 at different baud rates:
 { 9600 symbols.
 4800
 2400
 1200

$$\begin{aligned} & \downarrow \\ & 12 \\ & \text{UART} \\ & \downarrow \div 32. \\ & 11.0593 \text{ M} \\ & \hline 12 \times 32. \\ & 28,800 \text{ Hz} \end{aligned}$$

Timer clk freq = 28,800 Hz ⇒ for serial port programming.

$$\text{Timer clk period} = \frac{1}{28,800}$$

$$\text{Time taken to transmit a symbol of } n \text{ bits} = \frac{n}{28,800}$$

let Baud rate = 9600

$$\text{Time taken to transmit 1 symbol} = \frac{1}{9600}$$

$$\frac{1}{9600} = \frac{n}{28800}$$

$$n = \frac{28800}{9600} = 3_{10} = \text{FD}_{16} \leftarrow \text{Hexa decimal.}$$

let baud rate = 4800.

$$\frac{1}{4800} = \frac{n}{28800}$$

$$n = 6_{10} = \text{FA}$$

let baud rate = 2400

$$n = \frac{28800}{2400} = 12_{10} = \text{F4}$$

Let baud rate = 1200

$$n = \frac{28800}{1200} = 24_{10} = E8_{16}$$

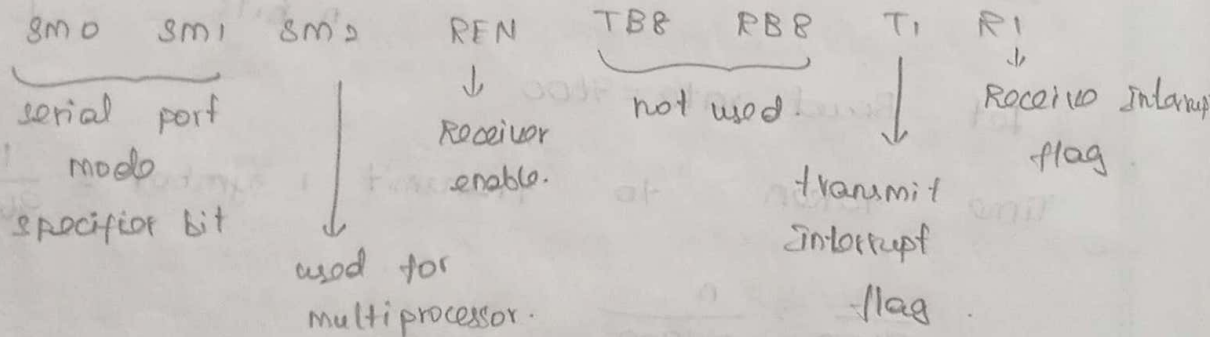
steps for transmitting 'A' bit serially.

1) Load TMOD register with the value 50H indicating timer 1 in mode 5.

2). TH1 is loaded with proper value to set baud rate for serial data transmission.

3). A small SCON loaded with 50H indicating serial mode 1 transmission.

SCON - serial control register (8 bit register).



TB8 - transmit Bit 8

RB8 - Receive bit 8

0 1 0 1 0 0 0 0 $\Rightarrow$ 50H

4). Timer 1 is set to 1

5). TI flag cleared.

6). The character byte to be transmitted serially is written into SBUF register.

4). To transfer next byte, goto step 5.

5). If TI flag is set, clear the TI flag

```
MOV TMOD, #20H
```

```
MOV TH, #FDH
```

```
MOV SCON, #50H
```

```
SETB TR1
```

```
AGAIN: MOV SBUF, #'A'
```

```
HERE: JNB TI, HERE.
```

```
CLR TI
```

```
SJMP AGAIN:
```

steps for receiving the data serially.

1) Load TMOD register with 20H indicating timer 1 mode.

2). TH1 is loaded with proper value to set baud rate for serial data transmission.

3). SCON loaded with 50H indicating serial mode 1 transmission.

4). Start timer.

5). Monitor RI flag bit using instruction.

JNB RI, target address.

6). When RI is raised, SBUF has byte. Its content moved to controller register [A].

```
MOV TMOD, #20H
```

```
MOV TH1, #FDH
```

```
MOV SCON, #50H
```

```
SETB TB.
```

```
HERE: JNB RI, HERE.
```

```
MOV A, SBUF.
```

```
CLR RI
```

```
SJMP HERE.
```

ISR → program associated with interrupt

[Interrupt service Routine]

Interrupt vector Table - It is the table in containing the starting address of ISR.

Available interrupts are:

*. Reset. - 8A 0000H

*. External hardware interrupt INTO. - 0003H

→ It is 12th pin multiplexed with Port 3.

*. Ext. Hardware interrupt INT1 - 0013H

software interrupts {

- *. Timer 0 overflow interrupt TFO - 000BH
- *. Timer 1 overflow interrupt TF1 - 001BH
- *. Serial communication interrupt - 0023H

[TI, RI]

Transmit interrupt

Receive Interrupt.

Reset:

Interrupt return - last line of ISR.

Start address - 0000H.

Ext. H/w interrupt INTO:

start address - 0003H is also called

vector address.

Interrupt.

* Signal sent by the device to the microcontroller.

ISR:

The program which needs to be executed in response to interrupt.

External Hardware Interrupt INT0 / INT1.

It has two types:

1. Level triggered interrupt.
2. Edge triggered interrupt.

①. Level triggered interrupt:

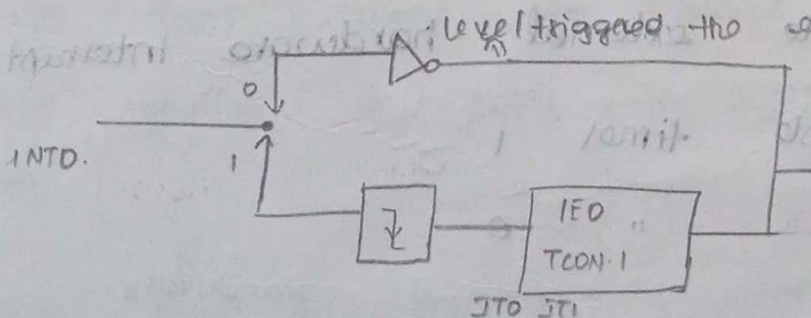
⇒ INT0 / INT1 - normally high

If it is not high, we apply

low level signal the microcontroller takes the low level signal as interrupt.

↳ it has to be removed before

level triggered the execution of return signal. otherwise another I/O will be generated.



edge triggered

②. Edge triggered interrupt:

TCON register we have bits called

IT0, IT1

after reset it will be level trigger
If we want to edge trigger. we
should follow this.

⇒ if we use
INT 0 ⇒ if we use,
INT 1

Make ITO → 1 make IT1 → 1
+ it in TCON register + it in TCON register

Enabling / Disable Interrupt: (IE)

IE → Interrupt enable register.

* This is 8 bit register.

EA - enable, all bit is 1

ET2 - enable timer 2 (in 8052 microcontroller)

ES - enable serially communication.

EX1 - enable external hardware interrupt 1

ET1 - enable timer 1

ET0 - " " 0

if we should make EA - 1 then
mandatory.

only other enable pin works.

EA	ET2	ES	ET1	EX1	ET0	EX0
----	-----	----	-----	-----	-----	-----

let the LED connected to P1.3 whenever interrupt signal occurs at INT1 LED has to turn ON & should remain ON for a fraction of seconds.

ORG 30H → start address of main program.
 MAIN: MOV IE, #84H. if initialize the IE logic

HERE: SJMP HERE

ORG 0013H → start address of INT1

SETB P1.3

BACK: MOV R3, #255

DJNZ R3, BACK.

CLR P1.3

RETI → last line of ISR is Return Interrupt.

Make EX1 → 1

already EA is 1.

10000100 → 84H.

SERIAL COMMUNICATION INTERRUPT:

eg: Assume data read from port 1 used as i/p.

write that data to port 2 continuously.

while giving a copy to serial communication port.

MAIN: MOV P1, #0FFH

MOV IE, #90H

MOV TMOD, #20H.

MOV TH1, #0FDH

MOV SCON, #50H

SETB TRI

* If we want to

use port as i/p

we should assign all

its bits FFH.

EA - ET2 ES ET1 EX1 ET0 EX0

1 0 0 1 0 0 0 0

= 90H

Serial port stops.

20 → timer 1 in mode 2.

BACK: MOV A, PI

SBUF → Serial Buffer.

MOV P2, A

MOV SBUF, A

SJMP BACK.

ORG 0023H → initialize IOP.

st. ad. of serial

communication: 0023H

JB TI, TRANS

MOV A, SBUF

If TJ → 1

CLR RI

come to CLR TI.

RETI

then return.

TRANS: CLR TI

RETI

TIMER INTERRUPT PROGRAMMING:

write a program that continuously get 8 bit data from port 0 and send to port 1, while simultaneously creating square wave of 200µs on P2.1 [use timer 0 in mode 1]

MAIN: MOV TMOD, #02H

MOV TH0, #A4H

MOV IE, #82H

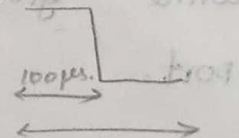
MOV P0, #0FFH

SETB TR0

BACK: MOV A, P0

MOV P1, A

SJMP BACK



$$\text{count} = \frac{100\mu\text{s}}{1.085\mu\text{s}}$$

$$= 92.165$$

$$256 - 92.165 = 163.835$$

$$163 \rightarrow A3$$

$$164 \rightarrow A4$$

ORG 000BH

CLR TFO

CPL P3.1

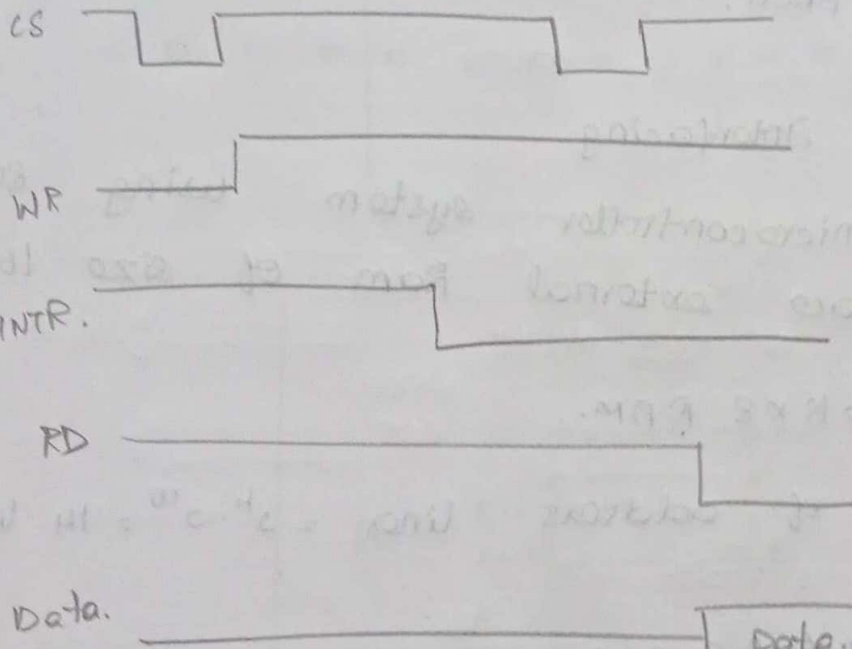
RETI

CPL → complementry

2/163
2/81-1
2/40-1
2/20-0
2/10-0
2/5-0
2/3-1
1-0

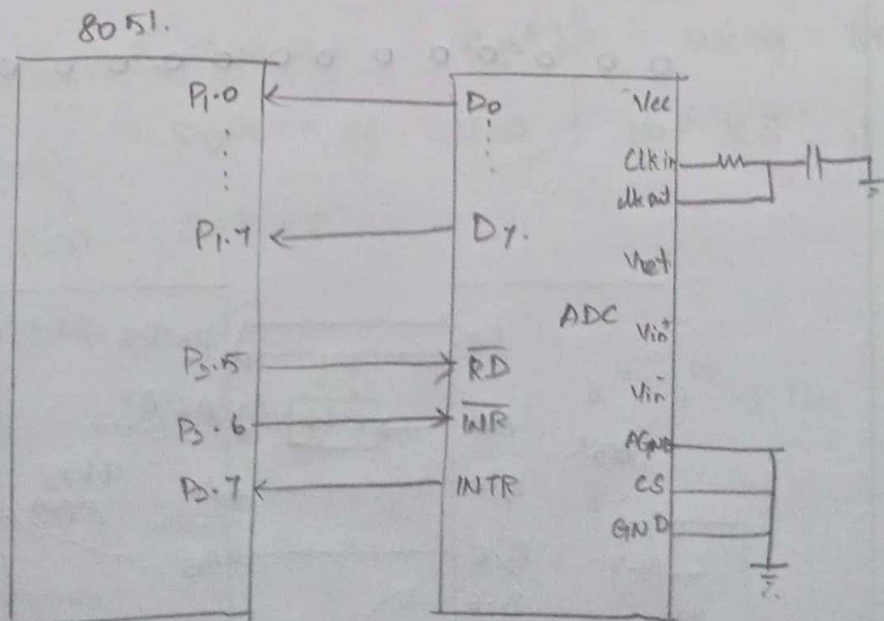
ADC

10100011
A 3



P₁ ← i/p port.

to issue
soc signal
and capture
Eoc signal.



program:

MOV P1, #FFH.

CLR P2.6

SETB P2.6

JB P2.7, HERE.

CLR P2.5

MOV A, P1

SJMP BACK.

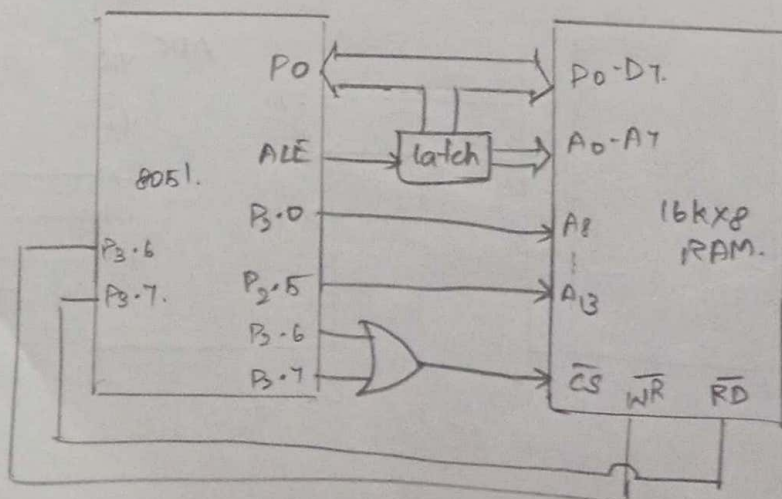
Memory Interfacing.

Design microcontroller system using 8051 to interface external Ram of size 16K x 8.

16K x 8 RAM.

no. of address lines = $2^4 \cdot 2^{10} = 14$ lines.

A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	3FFFH
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000H



At both
P2.6, P2.6 low
 $\overline{CS} \rightarrow$ active.

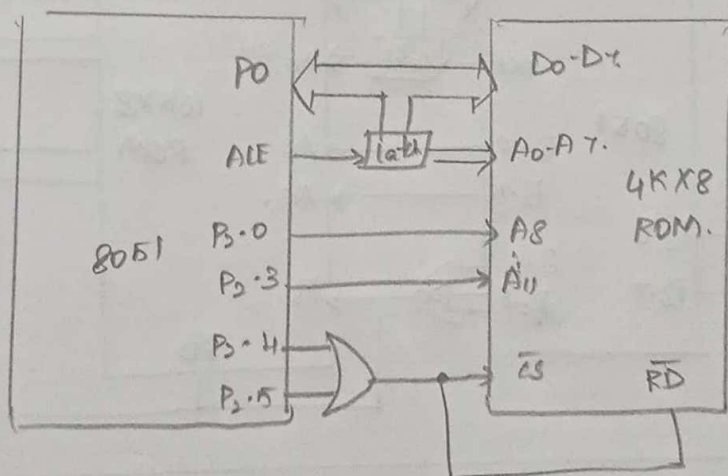
Design microcontroller system using 8051 to interface ROM of size $4K \times 8$.

No. of address lines = $2^4 \times 2^{10} = 12$

No. of data lines = 8.

A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

OFFFH
0000H



Design microcontroller system using 8051 to interface ROM of size $16K \times 8$, RAM of size $32K \times 8$.

$16K \times 8$ ROM.

No. of address lines = $2^4 \times 2^{10} = 14$

" " data " = 8.

$32K \times 8$ RAM.

No. of address lines = $2^5 \times 2^{10} = 15$ lines.

	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	
ROM	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	BFFFH
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	8000H
RAM	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	7FFFH
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000H

If A₁₅ → 1, ROM
 A₁₅ → 0, RAM.

